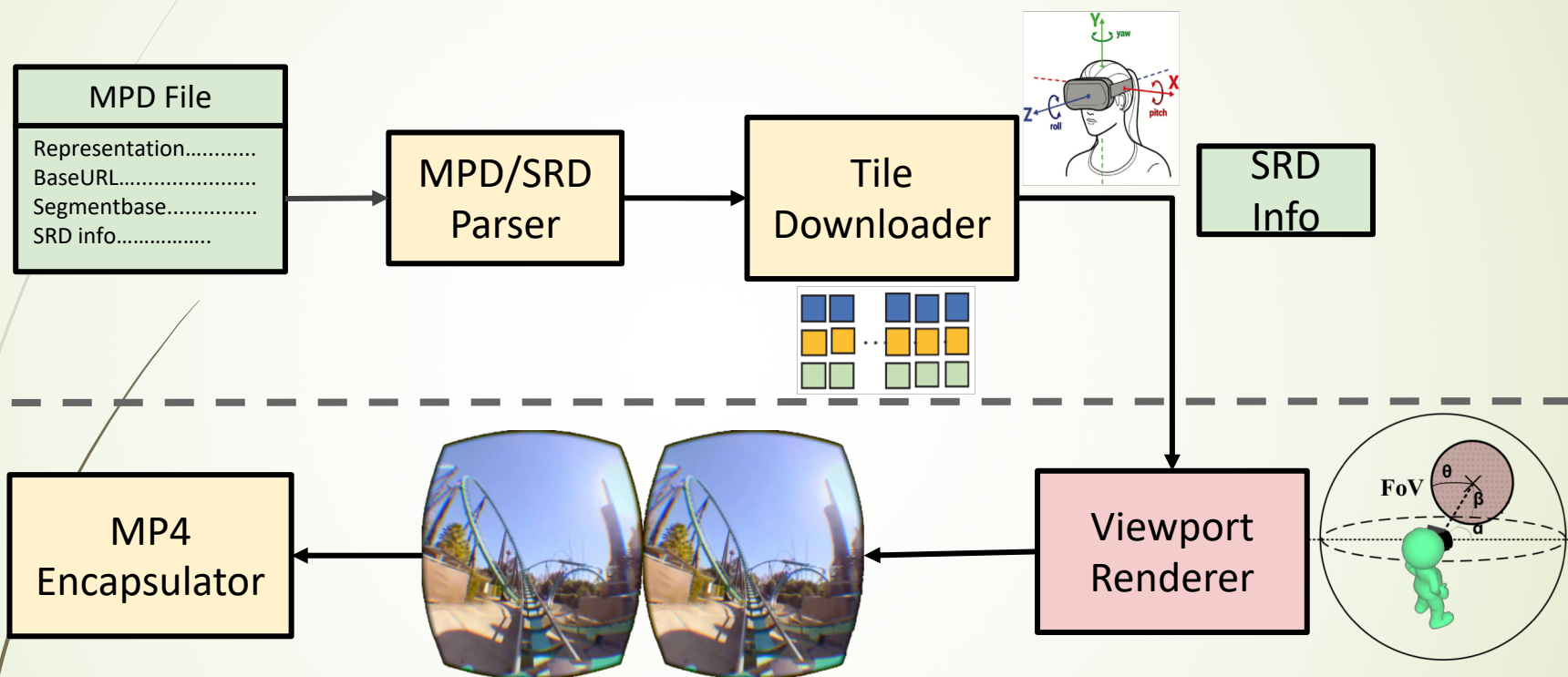


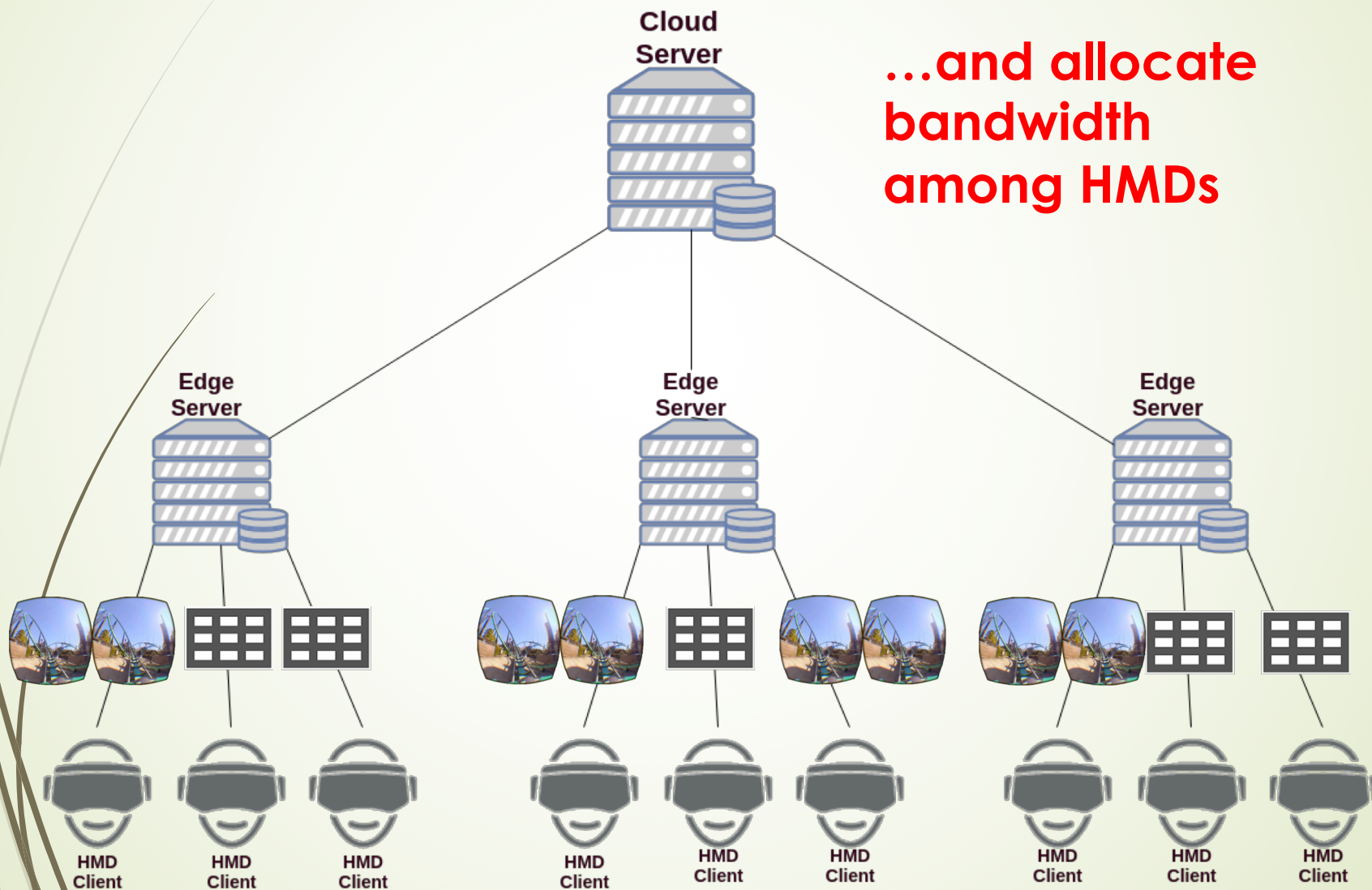
Viewport Rendering: Computation Offload to the Edge Server



- Parse MPD (with SRD info)
- Download tiles with different qualities
- Render user's viewport scene (FoV size, FPS, and resolution)
- Encapsulate HEVC bitstream into MP4 container

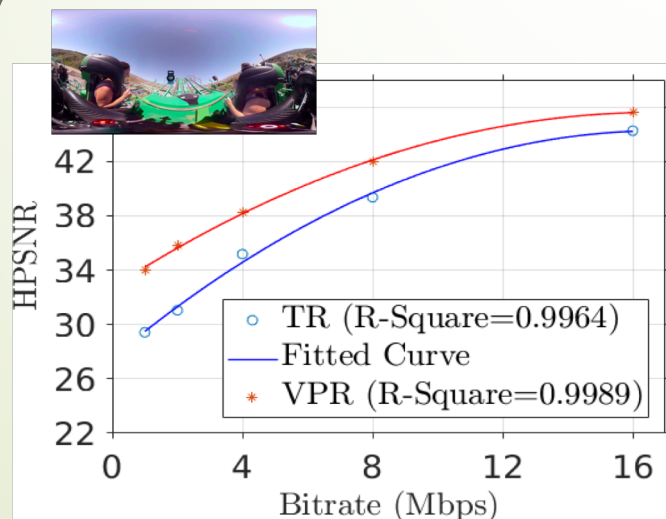
Choosing Between TR or VPR

...and allocate
bandwidth
among HMDs

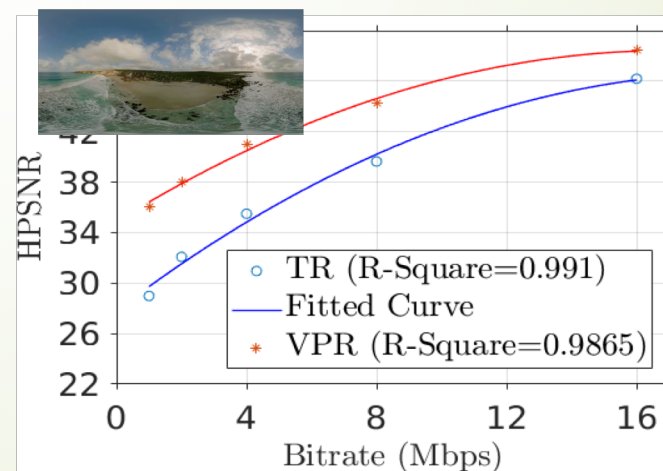


Resource Allocation

- Limited resources of edge server
 - Computing power
 - Network bandwidth
- Capitalize edge server to help HMDs for **maximizing the overall video quality** ← wisely assign bandwidth and computing power



Rate-Distortion Curves



Formulation

$$\text{maximize } 1/N \cdot \sum_{n=1}^N x_n (q'_n - q_n)$$

Objective: Maximize overall video quality (V-PSNR) improvement

$$s. t. \sum_{n=1}^N x_n \leq E$$

Avoid overloading the edge server

$$\sum_{n=1}^N [x_n (f_n^w f_n^h b_h) + (1 - x_n) (f_n^w f_n^h b_h + (T - f_n^w f_n^h) b_l)] \leq B$$

Bandwidth consumption of VPR

Bandwidth consumption of TR

$$x_n \in \{0, 1\}, \forall n = 1, 2, \dots, N$$

Outbound bandwidth of edge server doesn't exceed the limit

A (Greedy) Heuristic Algorithm

29

```
1: // We first initialize variables
2: for each  $n$  in  $N$  do
3:    $x[n] \leftarrow 0$ ;  $Qual[n] \leftarrow q'_n - q_n$ ;  $Band[n] \leftarrow \beta_n$ ;
    $Rati[n] \leftarrow (q'_n - q_n) / (\beta_n - \alpha_n)$ 
4: sort  $Qual[N]$ ,  $Band[N]$ ,  $Rati[N]$  in desc. order
5: // Case 1: ideal case, absolute quality boost
6: select first  $E$  HMD clients with the maximal  $Qual[n]$ 
7: for each  $e$  in  $E$  do
8:    $x[e] \leftarrow 1$ 
9:    $Band[e] \leftarrow \alpha[e]$ 
10: // Check if  $B$  is sufficient
11: if  $\text{sum}(Band[N]) \leq B$  then
12:   return  $x[N]$ 
13: // If  $B$  is insufficient, go to Case 2: per-unit-rate quality boost
14: select first  $E$  HMD clients with the maximal  $Rati[n]$ 
15: for each  $e$  in  $E$  do
16:    $x[e] \leftarrow 1$ 
17:    $Band[e] \leftarrow \alpha[e]$ 
18: // Check if  $B$  is sufficient
19: if  $\text{sum}(Band[N]) \leq B$  then
20:   return  $x[N]$ 
21: // If  $B$  is insufficient, go to Case 3: least bandwidth
22: select first  $E$  HMD clients with the maximal  $Band[n]$ 
23: for each  $e$  in  $E$  do
24:    $x[e] \leftarrow 1$ 
25:    $Band[e] \leftarrow \alpha[e]$ 
26: // Check if  $B$  is sufficient
27: if  $\text{sum}(Band[N]) \leq B$  then
28:   return  $x[N]$ 
29: return no feasible solution
```

Calculate Δq and saved bandwidth

Sort in desc. order

Max. video quality improvement

Max. ratio of quality improvement to bandwidth consumption

Max. bandwidth consumption

The Proposed Algorithm runs in **$O(N \log N)$**

Viewing Dataset

- 50 subjects
- Collect from HMDs while viewers are watching 360° videos
- Frame Capturer: GamingAnywhere[2]
- Sensor Logger: OpenTrack[3]

1: **no. frames, raw x, raw y, raw z, raw yaw, raw pitch, raw roll, cal. yaw, cal. pitch, cal. roll**

2: 00001, 16.458 ,30.032, -19.276, -9.661, 5.853, -3.068, -4.65473888889, 4.06641388889, -3.068

3: 00002, 16.458 ,30.032, -19.276, -9.661, 5.853, -3.068, -4.65473888889, 4.06641388889, -3.068

4: 00003, 16.449 ,30.02, -19.362, -9.763, 5.746, -3.184, -4.75673888889, 3.95941388889, -3.184

5: 00004, 16.449 ,30.02, -19.362, -9.763, 5.746, -3.184, -4.75673888889, 3.95941388889, -3.184

6: 00005, 16.433 ,30.007, -19.473, -9.676, 5.659, -3.308, -4.66973888889, 3.87241388889, -3.308

7: ...

[1] W. Lo, C. Fan, J. Lee, C. Huang, K. Chen, and C. Hsu, "360° video viewing dataset in head-mounted virtual reality," in Proc. of ACM International Conference on Multimedia Systems (MMSys'17), Taipei, Taiwan, June 2017, pp. 211–216.

[2] GamingAnywhere, <http://gaminganywhere.org/>

[3] OpenTrack, <https://github.com/opentrack/opentrack>